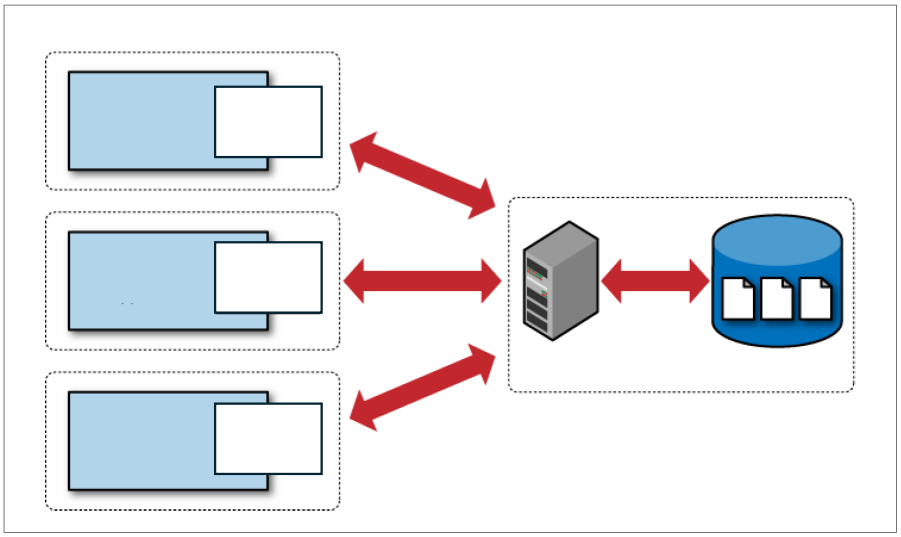
Reproducibility Engineering
Summer term 2026
Chair of Scalable Database Systems
Prof. Dr.-Ing. S. Scherzinger

UNIVERSITÄT PASSAU

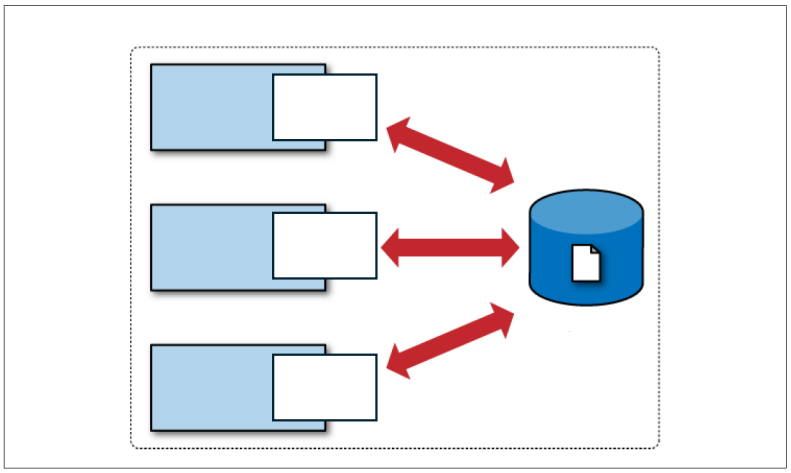
In-Class Exercise Sheet 6

1. **Database system architectures.** Label the database system architectures in the figures shown below. Choose appropriate captions and assign the terms below:

client host(s) / network / user application / SQLite library / DB client library / RDBMS server / DB file(s) / SQLite file(s)



(a) _____



(b) _____

Figures adapted from the textbook: Using SQLite, by Jay A. Kreibich, O'Reilly.

2. SQLite comes with several distinguishing features. Name these features and discuss their relevance w.r.t. reproducibility engineering.

   - _____: SQLite does not require a separate server process or system to operate. The SQLite library accesses its storage files directly.
   - _____: No server means no setup. Creating an SQLite database instance is as easy as opening a file.
   - _____: The entire database instance resides in a single cross-platform file, requiring no administration.
   - _____: A single library contains the entire database system, which integrates directly into a host application.
   - _____: The default build is less than a megabyte of code and requires only a few megabytes of memory. With some adjustments, both the library size and memory use can be significantly reduced.
   - _____: SQLite transactions are fully ACID-compliant, allowing safe access from multiple processes or threads.
   - _____: SQLite supports most of the query language features found in the SQL92 (SQL2) standard.
   - _____: The SQLite development team takes code testing and verification very seriously.

   Text originally from the textbook: Using SQLite, by Jay A. Kreibich, O'Reilly.

3. The SQLite website `https://www.sqlite.org/` states: "SQLite source code is in the public-domain and is free to everyone to use for any purpose."

   What are the consequences for reproducibility engineering?

4. SQLite is not a good choice when you need to handle

   - _____
   - _____
   - _____
   - _____
   - _____

Based on the textbook: Using SQLite, by Jay A. Kreibich, O'Reilly.

5. Complete the Docker Compose file below, for a small client/server database setup. The setup consists of two services:

   - a PostgreSQL database server
   - a benchmark client running `pgbench`.

   Out of convenience, we reuse the PostgreSQL image for the benchmark client because it contains the PostgreSQL client tools. However, in the benchmark client, no database server is started.

   The benchmark client connects to the database server through the Docker Compose network.

   Fill in the missing entries in the Compose file using the word bank below. Some entries are used more than once.

   Word bank:

   benchdb / db / lab / labpw / pgdata / 5433 / 5432 / postgres:16

Compose file (lines beginning with # are comments):

```
services:
   db:
      # The default behaviour of the PostgreSQL image is to start
      # a database server instance, so no additional boot
      # configuration is needed.
      image:  ____________

      # PostgreSQL uses the following environment variables
      # to create the initial database and user account.
      environment:
         POSTGRES_DB: ____________
         POSTGRES_USER: ____________
         POSTGRES_PASSWORD: ____________

      ports:
         # Exposes ports of the container over to the host
         # machine.
         #
         # Format:  "HOST_PORT:CONTAINER_PORT"
         # For this case, PostgreSQL listens on port 5432.
         # Other containers on this Docker network connect
         # to that port:  db:5432
         #
         # But the host machine cannot directly see
         # container ports.
         # So this exposes container port 5432 as host
         # port 5433:
         - "____________:____________"

      volumes:
         # Specifies how Docker volumes (defined in the
         # volumes section below) are mapped into this
         # container.
         #
         # For this case, the volume is mounted as
         # /var/lib/postgresql/data where PostgreSQL
         # stores tables, indexes, etc.
         - ____________:/var/lib/postgresql/data

   bench:
      # We reuse the same PostgreSQL image for the client
      # container.  Recall that the default behaviour of the
      # image is to start a server instance.
      # So to run it as a client instead, we need to override
```

```
      # its boot-up commands in the command section below.
      image:  ____________

      # This defines a dependency of the current container
      # on another one.
      depends_on:
         ____________:
            # For this case, the current container acts as
            # a client and depends on the server container.

      environment:
         # pgbench reads this as the PostgreSQL password.
         PGPASSWORD: ____________

      # Here, we can override the default boot-up behaviour
      # of the container to run pgbench instead of a (another)
      # database server.
      command:  >
         bash -c "
         # In the following commands, -h requires the
         # name of the server to connect to.
         # Docker resolves that name to the corresponding
         # container.
         #
         # Initialises pgbench tables:
         pgbench -i -h ____________ -U lab benchdb &&
         # Runs the benchmark for 30 seconds:
         pgbench -h ____________ -U lab -T 30 benchdb
         "

volumes:
   # Docker volumes are defined here.
   # Volumes are normal folders on the host machine
   # that Docker creates and manages automatically.
   #
   # Volumes are used to persist data across container
   # runs and also to share data between containers.
   #
   # This means, the command:
   # docker compose down
   # does not delete any files stored there when a
   # container is killed.
   ____________:
```

6. A student wants to record system information during a database experiment. They try to expose Linux memory information as a **PostgreSQL foreign table**:

```
CREATE FOREIGN TABLE experiment_meminfo (
    key   text,
    value text
)
SERVER local_files
OPTIONS (
    filename '/proc/meminfo',
    format 'csv',
    delimiter ':'
);
```

The statement succeeds. Which conclusion is justified?

- ◯ PostgreSQL has verified that `/proc/meminfo` exists.
- ◯ PostgreSQL has verified that the file can be read by the server process.
- ◯ PostgreSQL has verified that every line can be parsed into two columns.
- ◯ None of these options.

Which of the following statements are true? Check all that apply.

- ◯ Problems such as unreadable files or unexpected file format may only become visible when the table is queried.
- ◯ A query such as "`SELECT * FROM experiment_meminfo`" reads and parses the external file.
- ◯ Evaluating a selection such as "`WHERE key = 'MemTotal'`" can use an index on `/proc/meminfo`.
- ◯ For this example, scanning the complete source is usually cheap, because `/proc/meminfo` is small.
- ◯ For a very large CSV file, highly selective predicates in the `WHERE`-clause are guaranteed to make access efficient.
- ◯ Joining `experiment_meminfo` with PostgreSQL-internal base tables is possible and also carried out very efficiently.

7. Assume a large CSV file is as a foreign table `measurements`. Which query is likely to require scanning the entire file? Check all that apply.

- ◯ `SELECT * FROM measurements;`
- ◯ `SELECT * FROM measurements WHERE run_id = 42;`
- ◯ `SELECT COUNT(*) FROM measurements;`
- ◯ None of these options.